

# Git 초기화, GitHub 연동 및 배포 가이드

로컬 프로젝트 설정부터 GitHub Pages를 활용한 정적 웹사이트 배포까지의 전 과정

## 1. 로컬 Git 저장소 초기화 및 첫 커밋

웹 개발 프로젝트 폴더에서 변경 사항을 추적하고 버전을 관리하기 위해 Git을 초기화하고 첫 번째 커밋을 생성합니다.

1. **프로젝트 폴더 이동**: 터미널(Terminal) 또는 명령 프롬프트(CMD)를 열고 개발 중인 프로젝트의 최상위 디렉토리로 이동합니다.
2. **Git 저장소 초기화**: 다음 명령어를 입력하여 현재 폴더를 Git 저장소로 지정합니다.

```
git init
```

3. **.gitignore 파일 생성 (선택 사항)**: 불필요한 로그 파일, 빌드 결과물( `node_modules/` , `dist/` 등), 또는 보안이 필요한 환경 변수 파일이 Git에 추적되지 않도록 프로젝트 루트에 `.gitignore` 파일을 생성하고 대상을 작성합니다.
4. **스테이징 및 첫 커밋**: 프로젝트의 모든 파일을 스테이징 영역에 추가하고 첫 번째 버전을 기록합니다.

```
# 모든 파일을 스테이징 영역에 추가
git add .

# 첫 번째 커밋 생성
git commit -m "Initial commit"
```

## 2. GitHub 원격 저장소 생성 및 퍼블리시(Publish)

로컬에 기록된 커밋을 백업하고 공유할 수 있도록 GitHub 원격 저장소(Remote Repository)에 코드를 업로드합니다.

1. **GitHub 저장소 생성**: GitHub에 로그인한 후 오른쪽 상단의 **New** 버튼을 눌러 새 저장소를 생성합니다.
  - Repository name을 입력합니다.
  - GitHub Pages 배포를 위해 **Public**(공개)으로 설정하는 것이 좋습니다. (Private도 배포가 가능하나 설정이 다를 수 있음)
  - README, .gitignore, License 등은 로컬에서 이미 생성했으므로 체크하지 않고 **Create repository**를 클릭합니다.
2. **기본 브랜치 이름 변경**: 최근 Git 표준에 맞춰 기본 브랜치 이름을 `main` 으로 변경합니다.

```
git branch -M main
```

3. **원격 저장소 연결**: GitHub에서 제공하는 저장소 URL(HTTPS 주소)을 복사하여 로컬 저장소에 연결합니다.

```
git remote add origin https://github.com/사용자이름/저장소이름.git
```

4. **원격 저장소로 코드 발행(Push):** 로컬의 `main` 브랜치 내용을 원격 저장소로 전송합니다. `-u` 옵션을 주면 향후 `git push` 만 입력해도 자동으로 연결됩니다.

```
git push -u origin main
```

### 3. 웹 개발 및 배포 전 고려사항

GitHub Pages는 **정적 웹사이트 호스팅 서비스**입니다. PHP, Node.js(Express), Python(Django) 등 서버 사이드 코드가 상시 구동되어야 하는 웹 애플리케이션은 배포할 수 없으며, HTML, CSS, JavaScript 및 클라이언트 사이드 빌드 결과물만 배포할 수 있습니다.

#### 경로 설정 주의사항

GitHub Pages의 기본 주소 형식은 `https://사용자이름.github.io/저장소이름/` 꼴이 됩니다. 즉, 도메인의 루트 (`/`)가 아닌 하위 디렉토리 파일 구조를 가지므로 소스코드 내에서 자원을 불러올 때 주의가 필요합니다.

- 이미지, CSS, JS 파일을 불러올 때 절대 경로(예: `/images/logo.png`) 대신 **상대 경로**(예: `./images/logo.png`)를 사용하여 정상적으로 로드됩니다.
- 메인 페이지 역할을 하는 파일의 이름은 반드시 `index.html` 이어야 합니다.

### 4. GitHub Pages를 활용한 웹 사이트 배포

#### 방법 A: 순수 정적 파일(HTML/CSS/JS) 배포

별도의 빌드 과정이 없는 순수 퍼블리싱/웹 프론트엔드 프로젝트의 경우 GitHub 웹 인터페이스에서 간편하게 배포할 수 있습니다.

1. 코드가 업로드된 GitHub 저장소 화면으로 이동합니다.
2. 오른쪽 상단의 **Settings** 탭을 클릭합니다.
3. 왼쪽 사이드바 메뉴에서 **Code and automation** 섹션 아래의 **Pages**를 클릭합니다.
4. **Build and deployment** 설정 항목의 Source에서 `Deploy from a branch` 를 선택합니다.
5. Branch 설정에서 배포할 브랜치( `main` )와 폴더( `/root` )를 선택한 후 **Save** 버튼을 누릅니다.

#### 방법 B: 프레임워크 기반 프로젝트 배포 (Vite / React 등)

Vite, React, Vue 등 빌드 프로세스가 필요한 프로젝트는 빌드 결과물( `dist` 또는 `build` 폴더)을 배포해야 합니다. 가장 보편적인 방법은 `gh-pages` 패키지를 이용하는 것입니다.

1. 프로젝트에 배포용 패키지를 설치합니다.

```
npm install gh-pages --save-dev
```

2. `package.json` 파일을 열어 `homepage` 속성과 `scripts` 항목을 수정합니다.

```
{
  "homepage": "https://사용자이름.github.io/저장소이름",
  "scripts": {
    "predeploy": "npm run build",
    "deploy": "gh-pages -d dist"
  }
}
```

\* Vite 기준 빌드 폴더명은 `dist` 이며, CRA(Create React App) 등은 `build` 로 설정해야 합니다.

3. 터미널에서 다음 명령어를 실행하면 빌드와 배포가 자동으로 진행됩니다.

```
npm run deploy
```

#### 배포 결과 확인하기

설정 또는 스크립트 실행 후 1~5분 정도의 시간이 소요됩니다. 저장소의 **Actions** 탭에서 배포 진행 상황을 실시간으로 확인할 수 있으며, 완료되면 `https://사용자이름.github.io/저장소이름/` 주소로 접속하여 확인할 수 있습니다.